

---

# 第七章 ROS通信机制



# 本章提纲

---

基础概念

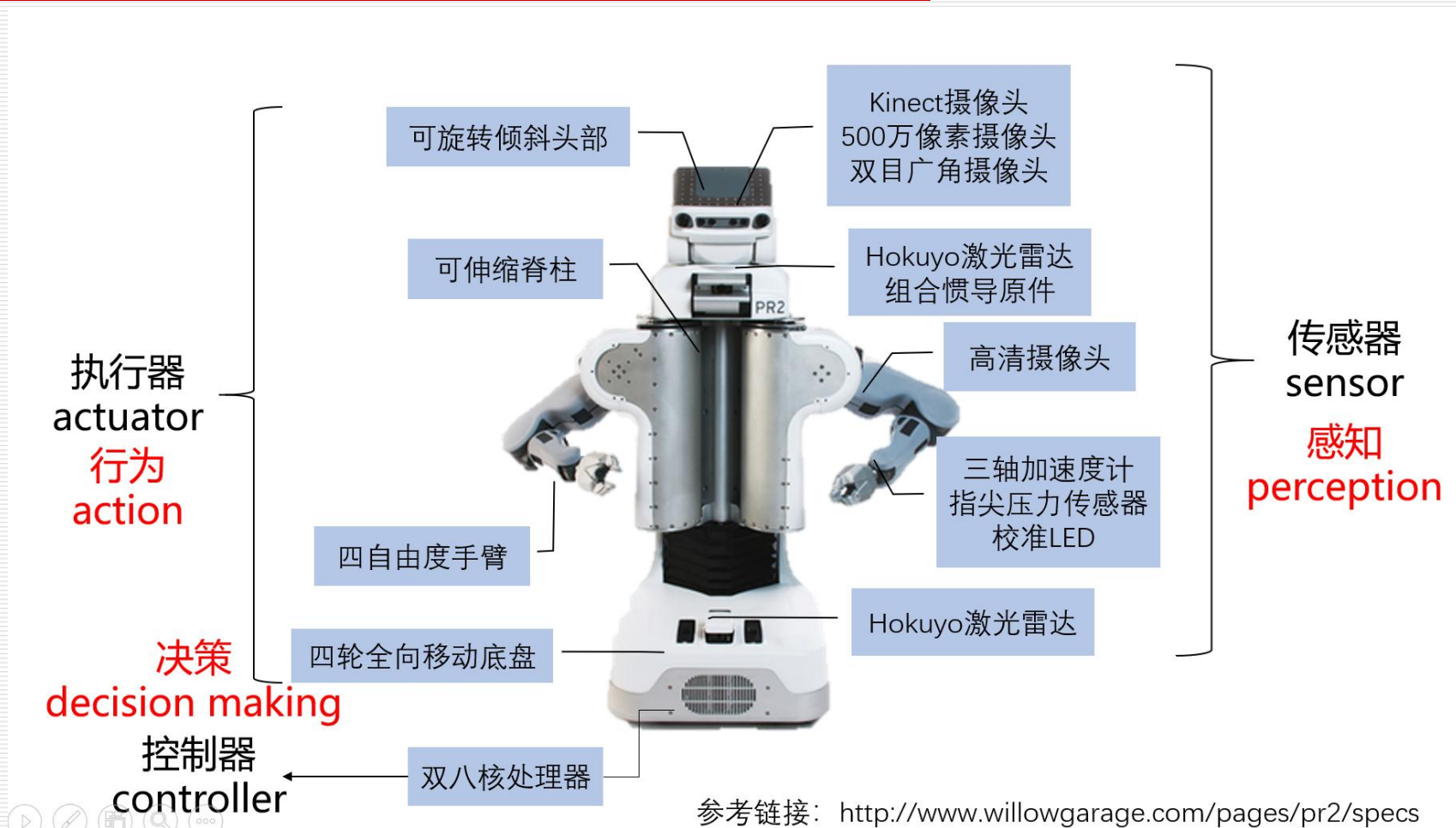
Topic

service

parameter server

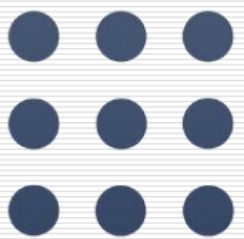
action

# 7.1 基础概念



## 7.1 基础概念

---

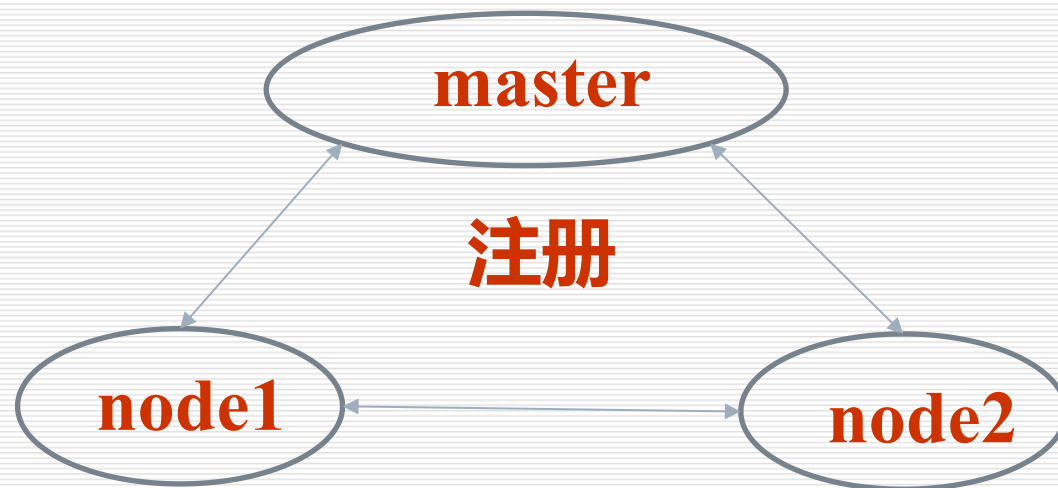
 ROS



## 7.1 基础概念

**master**

**每个node启动时都要向master注册**  
**管理node之间的通信**



## 7.1 基础概念

# Node

在ROS的世界里，最小的进程单元被称为节点（node）。

从**程序角度**来说，node就是一个可执行文件（通常为C++编译生成的可执行文件、Python脚本），可执行文件在运行之后就成为了一个进程（process），这个进程在ROS中就叫做节点。

从**功能角度**来说，通常一个node负责者机器人的某一个单独的功能。



## 7.1 基础概念

# Node

Master

由于机器人的元器件很多，功能庞大，因此实际运行时往往会运行众多的node，负责感知世界、控制运动、决策和计算等功能。那么如何合理的进行调配、管理这些node？



## 7.1 基础概念

# Node & Master

node首先在master处进行注册，之后master会将该node纳入整个ROS程序中。

node之间的通信也是先由master进行“牵线”，才能两两的进行点对点通信。

当ROS程序启动时，第一步首先启动master，由节点管理器处理依次启动node。





## 7.1 基础概念

### 启动Master

输入命令：\$ **roscore**。

此时ROS master启动，同时启动的还有 **rosout** 和 **parameter server**，其中 **rosout** 是负责日志输出的一个节点，其作用是告知用户当前系统的状态，包括输出系统的error、warning等等，并且将log记录于日志文件中；**parameter server** 即是参数服务器，它并不是一个node，而是存储参数配置的一个服务器文件。



# 7.1 基础概念

## 案例

```
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ roscore
... logging to /home/wh000/.ros/log/c6093f4c-bf52-11eb-8094-00dbdfd81788/roslaunch-wh000-Lenovo-ideapad-500S-15ISK-4069.lo
Checking log directory for disk usage. This may take awhile.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://wh000-Lenovo-ideapad-500S-15ISK:45537/
ros_comm version 1.12.14

SUMMARY
=====

PARAMETERS
* /roscdistro: kinetic
* /rosversion: 1.12.14

NODES

auto-starting new master
process[master]: started with pid [4080]
ROS_MASTER_URI=http://wh000-Lenovo-ideapad-500S-15ISK:11311/

setting /run_id to c6093f4c-bf52-11eb-8094-00dbdfd81788
process[rosout-1]: started with pid [4093]
started core service [/rosout]
```



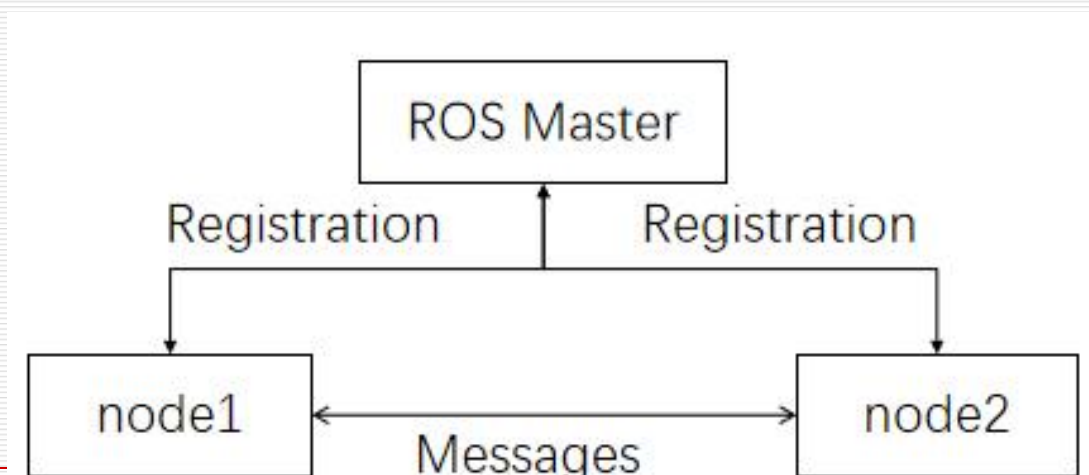
## 7.1 基础概念

### 启动Node

输入命令：\$ **roslaunch** **[--prefix cmd] [--debug] pkg\_name node\_name**  
**[ARGS]**。

roslaunch将会寻找对应包下的节点名的可执行程序，将可选参数ARGS传入。

启动后Master和Node关系如图：



```
roscore http://ubuntu:11311/
a@ubuntu: ~/catkin_temp$ roslaunch turtlesim turtlesim_node
[ INFO] [1742916069.210018163]: Starting turtlesim with node name /turtlesim
[ INFO] [1742916069.212073422]: Spawning turtle [turtle1] at x=[5.544445], y=[5.544445]
```





## roscnode

### 列出当前运行的node信息

```
$ roscnode list
```

### 显示某个node的详细信息

```
$ roscnode info [node_name]
```

### 结束某个node

```
$ roscnode kill [node_name]
```

```
a@ubuntu:~/catkin_temp$ roscnode list
/roscout
/turtlesim
a@ubuntu:~/catkin_temp$ roscnode info /turtlesim
-----
Node [/turtlesim]
Publications:
* /roscout [roscgraph_msgs/Log]
* /turtle1/color_sensor [turtlesim/Color]
* /turtle1/pose [turtlesim/Pose]

Subscriptions:
* /turtle1/cmd_vel [unknown type]

Services:
* /clear
* /kill
* /reset
* /spawn
* /turtle1/set_pen
* /turtle1/teleport_absolute
* /turtle1/teleport_relative
```



## 7.1 基础概念

### launch文件

- 机器人是一个系统工程，通常一个机器人运行操作时要开启很多个node，对于一个复杂的机器人的启动操作应该怎么做呢？
- roscore
- rosrune node1
- rosrune node2
- ...



## 7.1 基础概念

### launch文件

输入命令： `$ roslaunch pkg_name file_name.launch`。

`roslaunch`命令首先会自动进行检测系统的`roscore`有没有运行，也即是确认节点管理器是否在运行状态中，如果`master`没有启动，那么`roslaunch`就会 *首先启动master*，然后再按照`launch`的规则执行，把多个节点按照我们预先的配置启动起来。



# 7.1 基础概念

## launch写法

launch文件遵循着xml格式，是一种标签文本，包括以下标签：

|              |                         |
|--------------|-------------------------|
| <launch>     | <!--根标签-->              |
| <node>       | <!--需要启动的node及参数-->     |
| <include>    | <!--包含其他launch文件-->     |
| <machine>    | <!--指定运行的机器-->          |
| <env-loader> | <!--设置环境变量-->           |
| <param>      | <!--定义参数到参数服务器-->       |
| <rosparam>   | <!--启动yaml文件参数到参数服务器--> |
| <arg>        | <!--定义参数传入到launch文件中>   |
| <remap>      | <!--设定参数映射-->           |
| <group>      | <!--设定命名空间-->           |
| </launch>    | <!--根标签-->              |

参考链接:<http://wiki.ros.org/roslaunch/XML>





## 7.1 基础概念

### launch案例

<launch>

```
<node pkg="test" name="talker" type="talker" output="screen" />
```

```
<node pkg="test" name="listener" type="listener" output="screen" />
```

</launch>

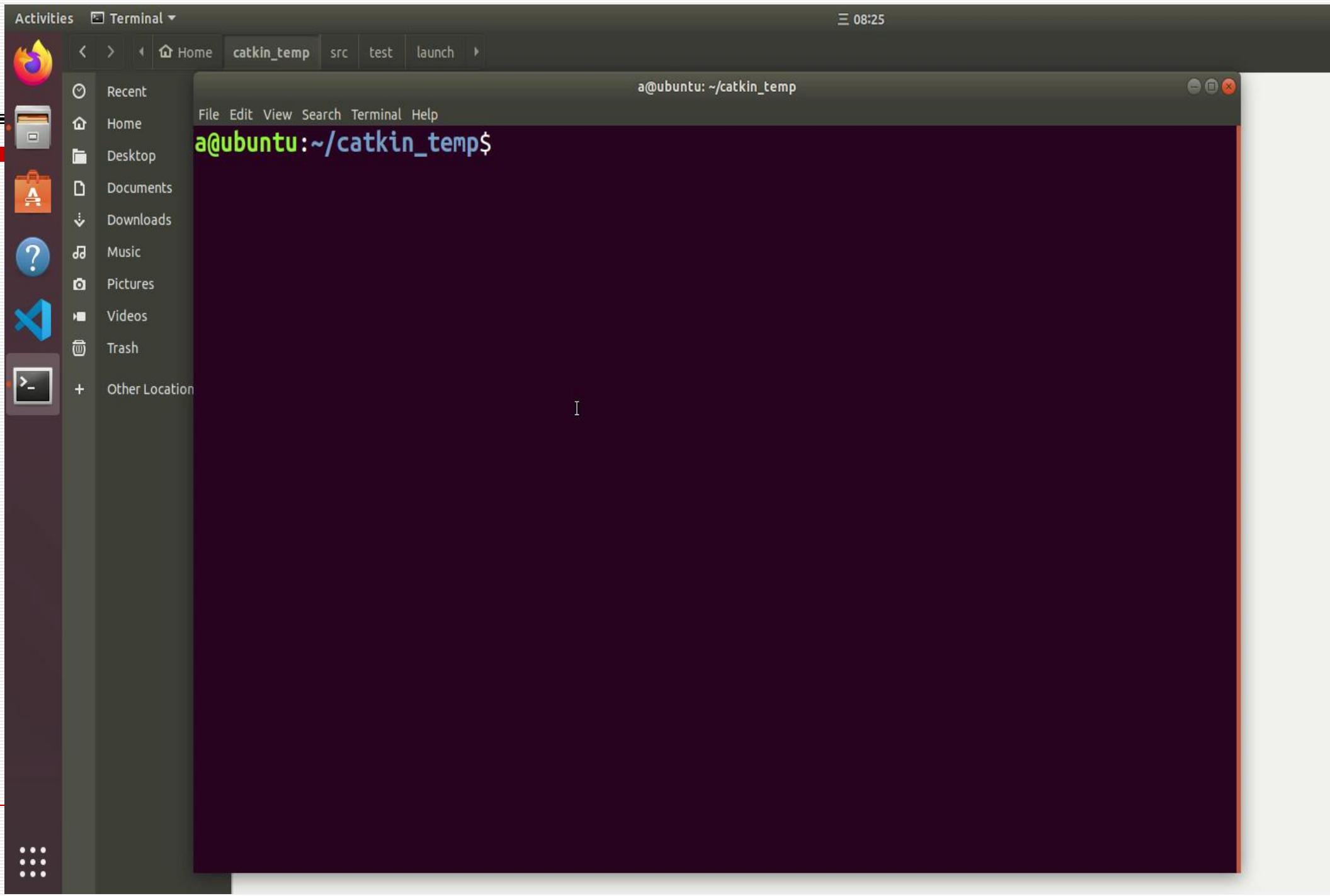
pkg指功能包

name是节点名称

type指节点的可执行文件名，通常是文件名(不包含文件扩展名)。如果python文件，通常要加.py后缀，如type="talker.py"



# launch案例



# 7.1 基础概念

## pr2启动运行的node

|                              |                           |
|------------------------------|---------------------------|
| pr2_description              | hokuyo_node               |
| pr2_machine                  | wifi_ddwrt                |
| pr2_ethercat                 | microstrain_3dmgx2_imu    |
| pr2_controller_manager       | wge100_camera             |
| pr2_controller_configuration | prosilica_camera          |
| pr2_calibration_controllers  | sound_play                |
| robot_mechanism_controllers  | joy                       |
| ethercat_trigger_controllers | std_srvs                  |
| pr2_head_action              | pr2_run_stop_auto_restart |
| pr2_gripper_action           | rosbag                    |
| joint_trajectory_action      | pr2_power_board           |
| single_joint_position_action | ocean_battery_driver      |
| tf2_ros                      | power_monitor             |
| robot_state_publisher        | pr2_computer_monitor      |
| robot_pose_ekf               | diagnostic_aggregator     |
| pr2_camera_synchronizer      | pr2_dashboard_aggregator  |
| stereo_image_proc            |                           |



**\$ roslaunch pr2\_bringup pr2.launch**

# 本章提纲

---

基础概念

Topic

service

parameter server

action

## 7.2 Topic

ROS的通信方式是ROS最为核心的概念，ROS系统的精髓就在于它提供的通信架构。

ROS中的通信方式中，Topic是常用的一种。Topic被称为话题（或者主题等等）。对于实时性、周期性的消息，使用topic来传输是最佳的选择。topic是一种点对点的单向通信方式，这里的“点”指的是node，也就是说node之间可以通过topic方式来传递信息。

消息的发送方叫做话题的发布者（Publisher）。

消息的接收方叫做话题的订阅者（Subscriber）

Master：提供名称解析服务，让发布者和订阅者能够找到对方。



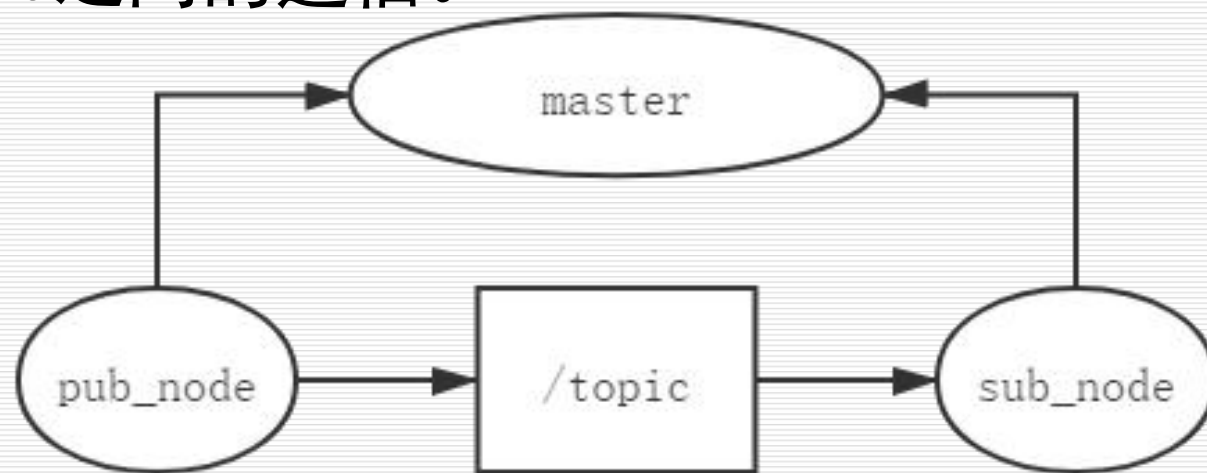
## 7.2 Topic

### 通信建立

首先，**publisher**节点和**subscriber**节点都要到节点管理器进行注册；

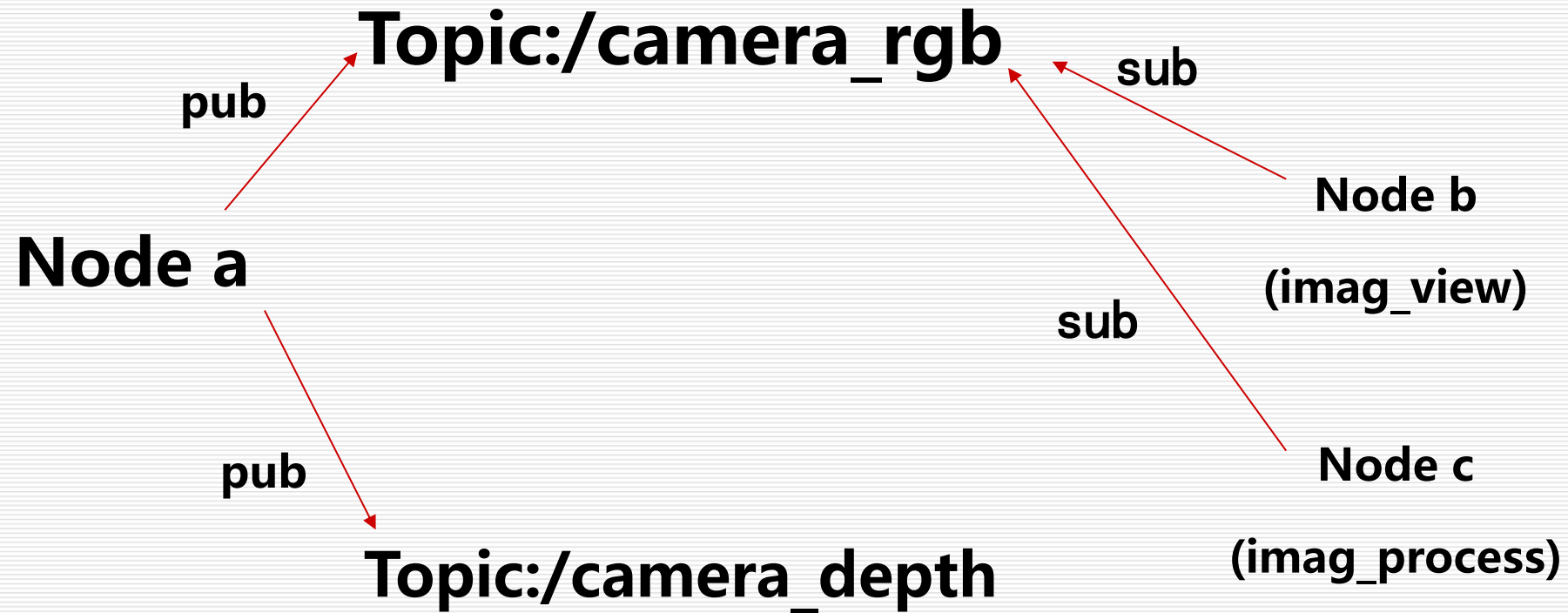
然后publisher会发布topic，subscriber在 master的指挥下会订阅该topic，从而建立起sub-pub之间的通信。

注意整个过程是**单向**的。



## 7.2 Topic

异步





## 7.2 Topic

---

### Subscriber处理

Subscriber接收消息会进行处理，一般这个过程叫做回调(Callback)。

所谓回调就是提前定义好了一个处理函数（写在代码中），当有消息来就会触发这个处理函数，函数会对消息进行处理。



## 7.2 Topic

---

### 特点1

Topic通信属于一种**异步**的通信方式。

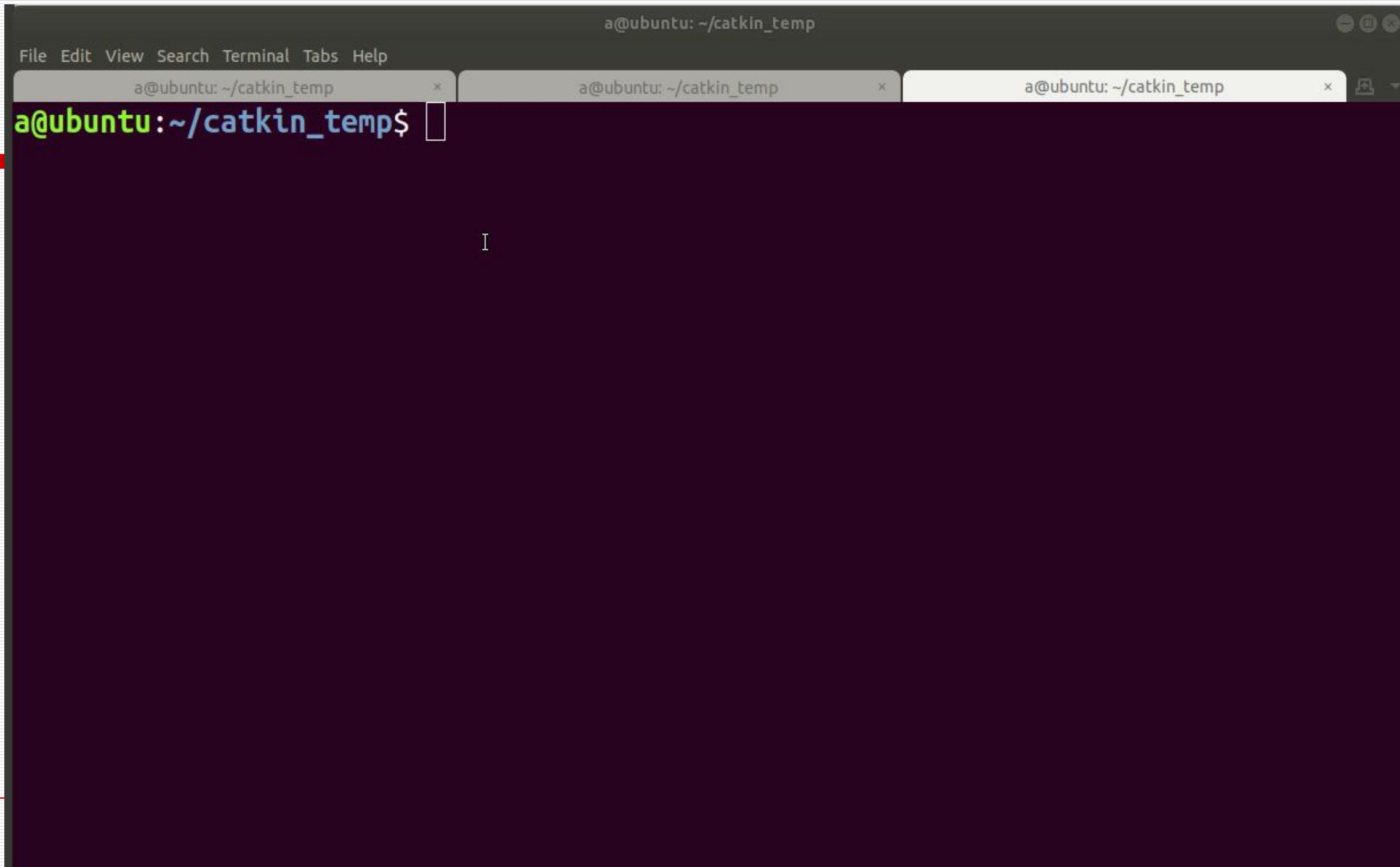
在node1每发布一次消息之后，就会继续执行下一个动作，至于消息是什么状态、被怎样处理，它不需要了解；而对于node2处理程序，它只管接收和处理消息，至于是谁发来的，它不会关心。所以node1、node2两者都是各司其职，不存在协同工作，我们称这样的通信方式是异步的。

### 特点2

ROS是一种**分布式**的架构，一个topic可以被多个节点同时发布，也可以同时被多个节点接收。



# 案例



## 7.2 Topic

### 操作命令

列出当前所有topic

```
$ rostopic list
```

显示某个topic的属性信息

```
$ rostopic info /topic_name
```

显示某个topic的内容

```
$ rostopic echo /topic_name
```

向某个topic发布内容

```
$ rostopic pub /topic_name ...
```

适当使用Table键补全命令



## 7.2 Topic

---

### 案例

输入命令：\$ `roscore`

\$ `roslaunch turtlesim turtlesim_node`

\$ `roslaunch turtlesim turtle_teleop_key`



## 7.2 Topic

**案例** (1) 显示所有话题 `rostopic list`

```
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rostopic list
/rosout
/rosout_agg
/turtle1/cmd_vel
/turtle1/color_sensor
/turtle1/pose
```

(2) 显示 `/turtle1/pose` 的内容

```
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rostopic echo /turtle1/pose
x: 5.80628585815
y: 5.8707280159
theta: 0.1599999996424
linear_velocity: 0.0
angular_velocity: 0.0
```



## 7.2 Topic

(3) 显示/turtle1/cmd\_vel的属性

```
a@ubuntu:~/catkin_temp$ rostopic info /turtle1/cmd_vel
Type: geometry_msgs/Twist

Publishers:
* /teleop_turtle (http://ubuntu:37537/)

Subscribers:
* /turtlesim (http://ubuntu:33751/)
```

可以看到/turtle1/cmd\_vel话题的消息类型为geometry\_msgs/Twist，订阅者为/turtlesim，发布者为/teleop\_turtle。

geometry\_msgs/Twist 是 ROS 里最常用的速度控制消息

绝大部分两轮差速小车 只需要这两个：

linear.x：前进速度（+ 向前，- 后退） angular.z：旋转速度（+ 左转，- 右转）



## 7.2 Topic

geometry\_msgs/Twist 是 ROS 里最常用的速度控制消息结构

linear.x 前后方向 (前进/后退)

linear.y 左右方向 (侧移)

linear.z 上下方向 (升降)

angular.x 翻滚 (很少用)

angular.y 俯仰 (很少用)

angular.z 自转 / 转弯 (最常用)

绝大部分两轮差速小车 只需要这两个:

linear.x: 前进速度 (+ 向前, - 后退)

angular.z: 旋转速度 (+ 左转, - 右转)

发布到话题: /cmd\_vel

```
#include <geometry_msgs/Twist.h>
```

```
// 创建速度消息
```

```
geometry_msgs::Twist cmd_vel;
```

```
// 前进
```

```
cmd_vel.linear.x = 0.5;
```

```
// 左转
```

```
cmd_vel.angular.z = 0.3;
```

```
// 发布到话题 /cmd_vel
```





## 7.2 Topic

### (4) 使用指定话题发布消息

```
a@ubuntu:~/catkin_temp$ rostopic pub -r 10 /turtle1/cmd_vel geometry_msgs/Twist "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```



这条指令表示/turtle1/cmd\_vel发布type为geometry\_msgs/Twist的消息，内容是小海龟按x线速度2，角速度1.8移动画圆。-r 10 表示循环发布，频率为10



## 7.2 Topic-Message

### 话题消息类型(Message)

Topic有很严格的格式要求，在传递消息时就必然要遵循ROS中定义好的格式。这种数据格式就是Message。

Message按照定义解释就是Topic内容的数据类型，也称之为Topic的格式标准。

### Message 信息基本类型

可以包括bool、int8、int16、int32、int64(以及uint)、float、float64、string、time、duration、header、可变长数组array[]、固定长度数组array[C]。



## 常见msg

`std_msg`中还包含一个特殊消息类型:**Head**, 表示包头, 它是一个结构体, 内置三个类型:

`uint32 seq`                      # 表示数据流的 sequenceID

`time stamp`                    # 表示时间戳

`string frame_id`               # 表示当前帧数据的帧头 (帧序号)

Head类型常用于记录每帧数据的时间和序列信息, 用于记录历史数据的情形。

## geometry\_msgs

是最常用的几何消息类型, 定义了描述机器人状态的各种类型, 比如点、速度、加速度、位姿等。

## 7.2 Topic-Message

### 常见msg

#### Vector3.msg

表示自由空间的三维向量

```
#文件位置:geometry_msgs/Vector3.msg
```

```
float64 x
```

```
float64 y
```

```
float64 z
```

#### Quaternion.msg

```
#消息代表空间中旋转的四元数
```

```
#文件位置:geometry_msgs/Quaternion.msg
```

```
float64 x
```

```
float64 y
```

```
float64 z
```

```
float64 w
```

ROS 中表示 机器人姿态 / 旋转 / 方向 的标准消息,  
表示 3D 空间中的旋转 (姿态)

ROS 里坐标变换、IMU、机器人姿态全用它



## 7.2 Topic-Message

### 常见msg

线速度和角速度Twist.msg

```
#定义空间中物体运动的线速度和角速度  
#文件位置: geometry_msgs/Twist.msg
```

```
Vector3 linear  
Vector3 angular
```

```
geometry_msgs::Twist twist;  
twist.linear.x = 0.5;  
twist.angular.z = 0.3;
```

**标准话题:**

**/cmd\_vel**

**机器人驱动订阅这个话题，接收 Twist 消息运动。**



## 7.2 Topic-Message

Pose. msg

geometry\_msgs/Pose  
位姿 (位置 + 姿态)

# 位置

Point position

# 姿态 (旋转/方向)

Quaternion orientation

# 位置坐标 (米)

float64 position.x

float64 position.y

float64 position.z

# 姿态 (四元数)

float64 orientation.x

float64 orientation.y

float64 orientation.z

float64 orientation.w

绝大多数两轮小车是 2D 平面运动，只需要：

position.x

position.y

orientation.z (yaw 方向角)

orientation.w

其余全部设为 0。





## 7.2 Topic-Message

### 操作命令

列出系统上所有msg

```
$ rosmmsg list
```

显示某个msg内容

```
$ rosmmsg show /msg_name
```

```
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rosmmsg list
actionlib/TestAction
actionlib/TestActionFeedback
actionlib/TestActionGoal
actionlib/TestActionResult
actionlib/TestFeedback
actionlib/TestGoal
actionlib/TestRequestAction
actionlib/TestRequestActionFeedback
actionlib/TestRequestActionGoal
actionlib/TestRequestActionResult
actionlib/TestRequestFeedback
actionlib/TestRequestGoal
actionlib/TestRequestResult
actionlib/TestResult
actionlib/TwoIntsAction
```

```
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rosmmsg show geometry_msgs/Vector3
float64 x
float64 y
float64 z
```



## 7.2 Topic-小结

---

Topic的通信方式是ROS中比较常见的**单向异步**通信方式

可以同时有多个subscribers

可以同时有多个publishers

一般实时而且周期性的

易用且高效.

然而有些时候单向的通信满足不了通信要求，比如当一些节点只是临时需要某些数据，如果用Topic通信方式时就会消耗大量不必要的系统资源，造成系统的低效率高功耗。





# 本章提纲

---

基础概念

Topic

service

parameter server

action

## 7.3 Service

---

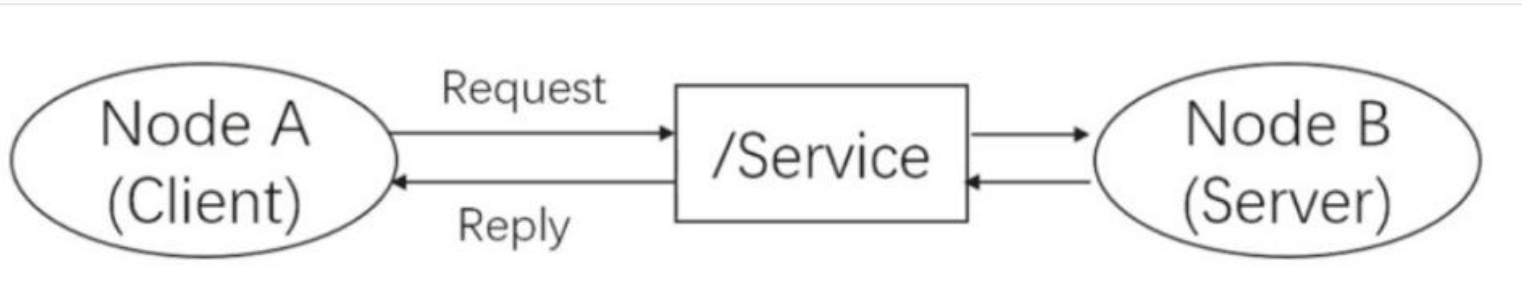
Service被称为服务。该方式方式在通信模型上与Topic做了区别。Service通信是**双向**的，它不仅可以发送消息，同时还会有反馈。所以service包括两部分，一部分是请求方（Client），另一部分是应答方/服务提供方（Server）。这时请求方（Client）就会发送一个request，要等待server处理，反馈回一个reply，这样通过类似“请求-应答”的机制完成整个服务通信。



## 7.3 Service

### 通信方式

Node B是server（应答方），提供了一个服务的接口，叫做 /Service，我们一般都会用string类型来指定service的名称，类似于topic。Node A向Node B发起了请求，经过处理后得到了反馈。



Service是**同步通信**方式，所谓同步，Node A发布请求后会在原地等待reply，直到 Node B处理完了请求并且完成了reply，Node A才会继续执行。Node A等待过程中，是处于阻塞状态的。这样的通信模型没有频繁的消息传递，没有冲突与高系统资源的占用，只有接受请求才执行服务，简单而且高效。

## 7.3 Service

### Topic VS Service

|      | Topic                     | Service            |
|------|---------------------------|--------------------|
| 通信方式 | 异步通信                      | 同步通信               |
| 实现原理 | TCP/IP                    | TCP/IP             |
| 通信模型 | Publish-Subscribe         | Request-Reply      |
|      | Publisher-Subscriber(多对多) | Client-Server(多对一) |
|      | 接收者收到数据会回调(Callback)      | 远程过程调用(RPC)服务器端的服务 |
| 应用场景 | 连续、高频的数据发布                | 偶尔调用的功能/具体的任务      |
| 举例   | 激光雷达、里程计发布数据              | 开关传感器、拍照、逆解计算      |

## 7.3 Service

**my\_pkg/srv/DetectHuman.srv**

**srv**

**Service通信的数据格式**

**定义在\*.srv文件中**

**srv文件格式固定，第一行是请求的格式，中间用---隔开，第三行是应答的格式。**

以DetectHuman.srv文件为例，该服务例子取自OpenNI的人体检测ROS软件包。它是用来查询当前深度摄像头中的人体姿态和关节数的。请求为是否开始检测，应答为一个数组，数组的每个元素为某个人的姿态（HumanPose）。而对于人的姿态，其实是一个msg，所以srv可以嵌套msg在其中

[https://blog.csdn.net/qq\\_30193419/article/details/111867500](https://blog.csdn.net/qq_30193419/article/details/111867500) **ROS中的数据类型扩展**

```
bool start_detect
---
my_pkg/HumanPose[] pose_data
```

**my\_pkg/msg/HumanPose.msg**

```
std_msgs/Header header
string uuid
int32 number_of_joints
my_pkg/JointPos[] joint_data
```

**my\_pkg/msg/JointPos.msg**

```
string joint_name
geometry_msgs/Pose pose
float32 confidence
```

## 7.3 Service

修改package.xml，添加依赖

```
<build_depend>message_generation</build_depend>  
<run_depend>message_runtime</run_depend>
```

修改CMakeList.txt，添加

```
find_package(... roscpp rospy std_msgs message_generation)  
  
catkin_package(  
  ...  
  CATKIN_DEPENDS message_runtime ...  
  ...)  
  
add_message_files(  
  FILES  
  DetectHuman.srv  
  HumanPose.msg  
  JointPos.msg)  
  
generate_messages(DEPENDENCIES std_msgs)
```

.srv文件建立后，需要修改cmakelist以及package.xml文件，添加的这些内容指定了srv或者msg在编译或者运行中需要的依赖。

message\_generation的主要作用是将ROS中的消息定义文件（.msg文件）转换为不同编程语言的代码。

message\_runtime运行时消息处理工具

## 7.3 Service

### 操作命令

| rosservice 命令          | 作用             |
|------------------------|----------------|
| <b>rosservice list</b> | 显示服务列表         |
| <b>rosservice info</b> | 打印服务信息         |
| <b>rosservice type</b> | 打印服务类型         |
| <b>rosservice find</b> | 按服务类型查找服务      |
| <b>rosservice call</b> | 使用所提供的args调用服务 |
| <b>rosservice args</b> | 打印服务参数         |
| ...                    | ...            |





## 7.3 Service

### 案例 (1) 显示服务话题列表

```
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rosservice list
/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/spawn
/teleop_turtle/get_loggers
/teleop_turtle/set_logger_level
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
```





## 7.3 Service

### (2) 发布服务话题，修改颜色

```
wh000@wh000-Lenovo-ideapad-500S-15ISK: ~$ rosservice list
usage. Usage is <1GB.
ip://wh000-Lenovo-ideapad-500S-15ISK:45195/

/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/spawn
/teleop_turtle/get_loggers
/teleop_turtle/set_logger_level
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level

wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rosservice info /turtle1/set_pen
Node: /turtlesim
URI: rosrpc://wh000-Lenovo-ideapad-500S-15ISK:57671
Type: turtlesim/SetPen
Args: r g b width off
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ rosservice call /turtle1/set_pen 255 0
0 5 0
```

设置小乌龟行走路线的颜色

- 1 `rosservice call /turtle1/set_pen "{r: 255, g: 0, b: 0, width: 10, 'off': 0}"`
- 2
- 3 `r, g, b`控制画笔的颜色值。取值范围为0-255.
- 4 `width`控制的是画笔的粗细。
- 5 `off`表示是否使用画笔，0代表是使用，1为不使用。不使用时看不见行走的路线





```
wh000@wh000-Lenovo-ideapad-500S-15ISK: ~/myspace
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ cd my
myspace/ mytest/
wh000@wh000-Lenovo-ideapad-500S-15ISK:~$ cd myspace/
wh000@wh000-Lenovo-ideapad-500S-15ISK:~/myspace$ roslaunch learnin
client
```

#### SUMMARY

=====

#### PARAMETERS

```
le * /rostdistro: kinetic
pa * /rosversion: 1.12.14
```

#### NODES

auto-starting new master

process[**master**]: started with pid [8091]

ROS\_MASTER\_URI=http://wh000-Lenovo-ideapad-500S-15ISK:11311/

setting /run\_id to 556db0b4-bf5d-11eb-8094-00dbdfd81788

process[**rosout-1**]: started with pid [8104]

started core service [/rosout]

^C[**rosout-1**] killing on exit

[**master**] killing on exit

shutting down processing monitor...

... shutting down processing monitor complete

done

wh000@wh000-Lenovo-ideapad-500S-15ISK:~\$

## 7.3 Service

### 操作命令

| rossrv 命令             | 作用      |
|-----------------------|---------|
| <b>rossrv show</b>    | 显示服务描述  |
| <b>rossrv list</b>    | 列出所有服务  |
| <b>rossrv package</b> | 列出包中的服务 |
| ...                   | ...     |

**rosservice list:** 查看正在运行的服务（运行时）

**rossrv list:** 查看所有服务类型（系统文件）



## 7.3 Service

### 列出所有的srv消息

```
1 | rossrv list
2 | control_msgs/QueryCalibrationState
3 | control_msgs/QueryTrajectoryState
4 | control_toolbox/SetPidGains
5 | controller_manager_msgs/ListControllerTypes
```

### •列出包含服务消息的所有包

```
1 | rossrv packages
2 | control_msgs
3 | control_toolbox
4 | controller_manager_msgs
```

### •列出某个包下的所有服务消息

```
1 | rossrv package server_client
2 | server_client/add
```

## 7.3 Service-小结

---

### Service

用于那些快速停止的程序，如查询一个node 的状态，或是做数学运算  
Service通信是双向的，请求方（Client）就会发送一个request，要等待server处理，反馈回一个reply，这样通过类似“请求-应答”的机制完成整个服务通信。

用于那些快速停止的程序，如查询一个node 的状态，或是做数学运算，比如机械臂的运动学反解IK（inverse kinematic）。它不应当被用于那些需要长时间运行的进程中，



# 本章提纲

---

基础概念

Topic

service

parameter server

action



## 7.4 Parameter server

Parameter server被称为参数服务器。与前两种通信方式不同，参数服务器也可以说是特殊的“通信方式”。特殊点在于参数服务器是**存储各种参数（一般是静态参数）的字典**。

参数服务器使用互联网传输，在节点管理器中运行，实现整个通信过程。

| Key   | /roscdistro | /rosversion | /use_sim_time | ... |
|-------|-------------|-------------|---------------|-----|
| Value | 'kinetic'   | '1.12.7'    | true          | ... |

**# 列出所有参数**

**roscparam list**

**# 查看某个参数的值**

**roscparam get /max\_speed**

**# 设置参数**

**roscparam set /max\_speed 1.0**





## 7.4 Parameter server

---

### 维护方式

参数服务器的维护方式非常的简单灵活，总的来讲有**三种**方式：

- 命令行维护
- launch文件内读写（YAML文件）
- node源码



## 7.4 Parameter server

### (1) 命令行维护

#### rosparam

| rosparam 命令                                   | 作用      |
|-----------------------------------------------|---------|
| <b>rosparam set param_key<br/>param_value</b> | 设置参数    |
| <b>rosparam get param_key</b>                 | 显示参数    |
| <b>rosparam load file_name</b>                | 从文件加载参数 |
| <b>rosparam dump file_name</b>                | 保存参数到文件 |
| <b>rosparam delete</b>                        | 删除参数    |
| <b>rosparam list</b>                          | 列出参数名称  |



启动 master，开启终端，输入 roscore，运行 turtle 节点，  
roslaunch turtlesim turtlesim\_node，新开启一个终端，输入

```
ra@ubuntu:~$ rosparam list
/background_g
/background_r
/rosdistro
/roslaunch/uris/host_ubuntu__38159
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
```

有三个参数是修改背景颜色的。

rosparam set background\_r 250

## 7.4 Parameter server

### (2) launch文件读写

launch文件中有很多标签，而与参数服务器相关的标签只有两个，一个是 `<param>`，另一个是 `<rosparam>`。

`param`只对一个参数进行操作。`rosparam`可以对多个参数进行操作，前提时把这些参数放到`.yaml`文件中。

```
<param name="name" value="ture"/>
```

```
<rosparam file="param.yaml" command="load"/>
```

### YAML格式举例

```
name: 'Zhangsan'
age: 20
gender: 'M'
score: {Chinese: 80, Math: 90}
score_history: [85,82,88,90]
```

### load和dump文件

load和dump（存取）文件需要遵守YAML格式。



```
<launch>
  <!-- 读取机器人模型参数-->
  <param name="robot_description" command="$(find xacro)/xacro.py $(find robot_sim_demo)/urdf/robot.xacro" />

  <!--在Gazebo中启动机器人模型-->
  <node name="urdf_spawner" pkg="gazebo_ros" type="spawn_model" respawn="false" output="screen"
    args="-urdf -x $(arg x) -y $(arg y) -z $(arg z) -Y $(arg yaw) -model xbot2 -param robot_description"/>

  <!--把关节控制的配置信息读到参数服务器-->
  <rosparam file="$(find robot_sim_demo)/config/xbot2_control.yaml" command="load"/>

  <!--启动关节控制器-->
  <node name="spawner" pkg="controller_manager" type="spawner" respawn="false" output="screen" ns="/xbot2"
    args="joint_state_controller yaw_platform_position_controller pitch_platform_position_controller"/>

  <!--将关节状态转换为TF变换 -->
  <node name="robot_state_publisher" pkg="robot_state_publisher" type="robot_state_publisher" ns="/xbot2"
    respawn="false" output="screen">
    <param name="publish_frequency" value="100.0"/>
  </node>
</launch>
```

## 7.4 Parameter server

### (3) node源码

除了上述最常用的两种读写参数服务器的方法，还有一种就是修改ROS的源码，也就是利用API来对参数服务器进行操作。但是此方法较为复杂，因此使用较少。

```
#include <ros/ros.h>
std::string distro;
bool use_sim;
ros::param::get("/rostdistro", distro);
ros::param::get("/use_sim_time", use_sim);
```





未命名文件夹

CARLA0.8.4.txt



WPS 2019

wh000@wh000-Lenovo-ideapad-500S-15ISK: ~/myspace

wh000@wh000-Lenovo-ideapad-500S-15ISK:~/myspace\$ rosrn turtles

SUMMARY

=====

PARAMETERS

\* /rostdistro: kinetic  
\* /rosversion: 1.12.14

NODES

auto-starting new master

process[**master**]: started with pid [9937]

ROS\_MASTER\_URI=http://wh000-Lenovo-ideapad-500S-15ISK:11311/

setting /run\_id to 410b5a30-bf5e-11eb-8094-00dbdfd81788

process[**rosout-1**]: started with pid [9950]

started core service [/rosout]

^C[rosout-1] killing on exit

[master] killing on exit

shutting down processing monitor...

... shutting down processing monitor complete

done

wh000@wh000-Lenovo-ideapad-500S-15ISK:~\$ roscore



## 7.4 Parameter server

---

Parameter server的主要作用是方便参数的存储，可以通过命令实时增删查改，也可通过文件或者源码方式配置多个参数，然后全局共享。



# 本章提纲

---

基础概念

Topic

service

parameter server

Action

## 7.5 Action

---

Action是ROS中一个很重要的**请求响应机制**的通信方式，类似Service通信机制，Action主要弥补了Service通信的不足。

当机器人执行一个长时间的任务时，如果利用Service通信方式，那么 node 会很长时间接受不到反馈的reply，致使通信受阻。而Action则可以比较适合实现长时间的通信过程，Action通信过程可以随时被查看过程进度，也可以终止请求，这样的特性，使得它在一些特别的机制中拥有很高的效率。

Action可以通过反馈获取动作任务的**进度**，还可以发送请求，终止动作任务。



## 7.5 Action

---

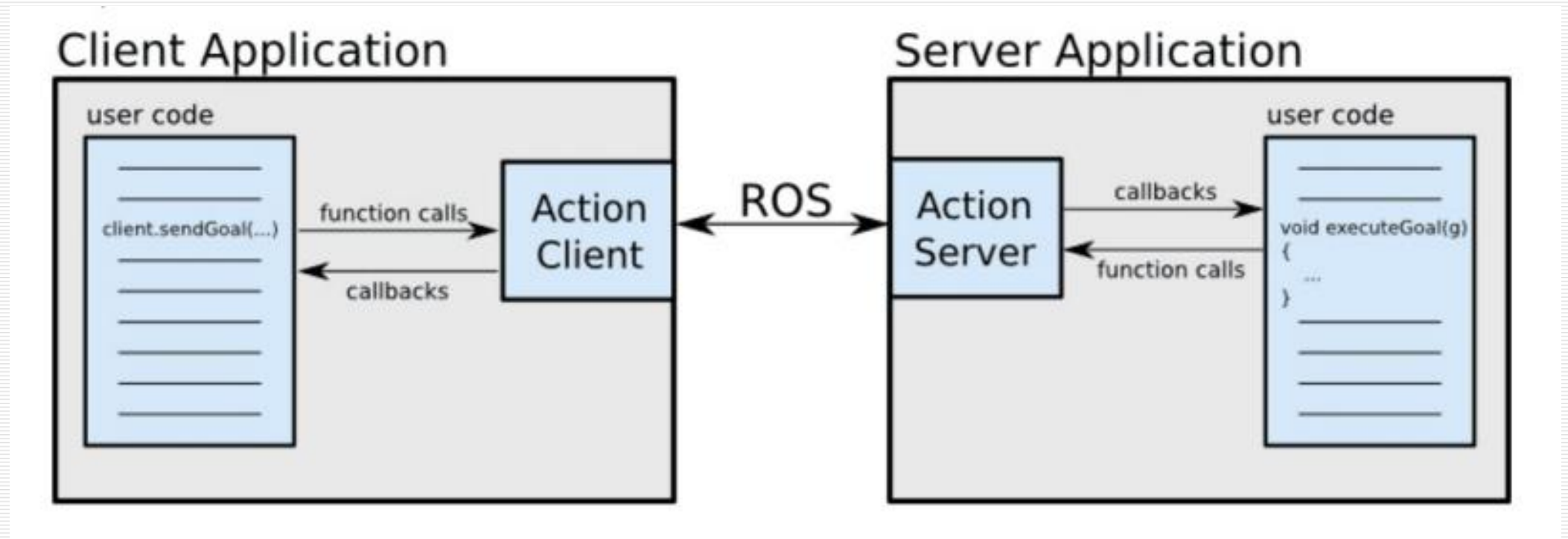
### 通信原理

Action的工作原理是client-server模式，也是一个**双向**的通信模式。通信双方在ROS Action Protocol下通过**消息**进行数据的交流通信。client和server为用户提供一个简单的API来请求目标（在客户端）或通过函数调用和回调来执行目标（在服务器端）。



## 7.5 Action

### 结构示意图

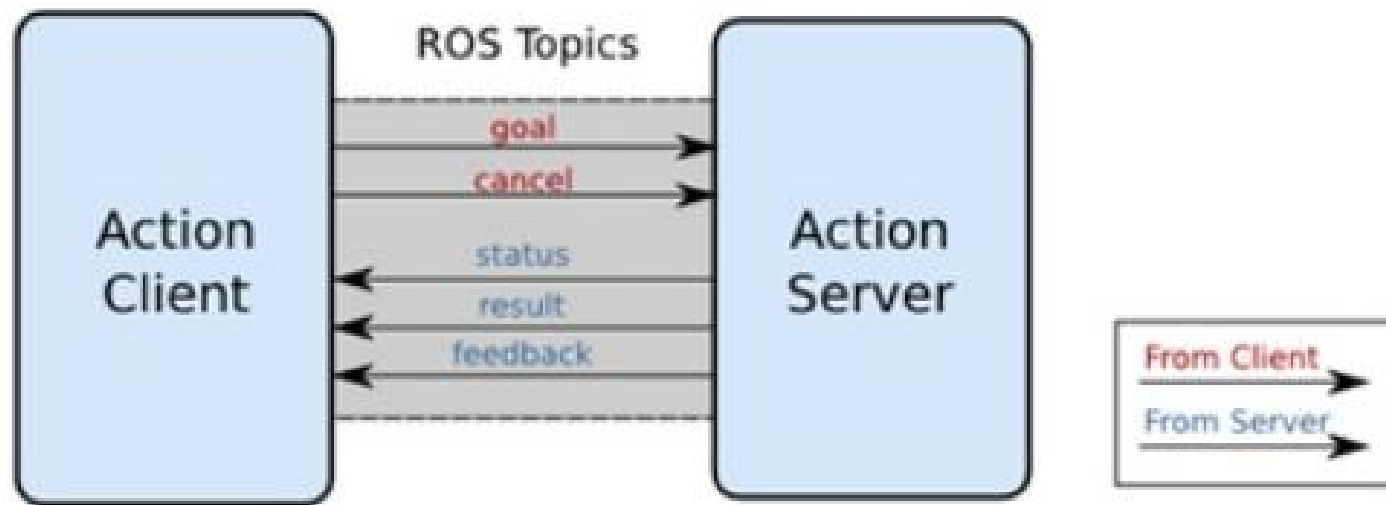


## 7.5 Action

### 通信接口

客户端会向服务器发送目标指令和取消动作指令，服务器则可以给客户端发送**实时**的状态信息、结果信息、**反馈**信息等等，从而完成了 service 没法做到的部分。

Action Interface



## 7.5 Action

### action 规范

- \* **目标** 机器人执行一个动作，应该有明确的移动目标信息，包括一些参数的设定，方向、角度、速度等等，从而使机器人完成动作任务。
- \* **反馈** 在动作进行的过程中，应该有实时的状态信息**反馈**给服务器的实施者，告诉实施者动作完成的状态，可以使实施者作出准确的判断去**修正**命令。
- \* **结果** 当运动完成时，动作服务器把本次运动的结果数据发送给客户端，使客户端得到本次动作的全部信息，例如可能包含机器人的运动时长，最终位姿等等。





## 7.5 Action

### Action文件格式

Action规范文件的后缀名是.action，如：

#### ./action/DoDishes.action

```
# Define the goal
uint32 dishwasher_id
# Specify which dishwasher we want to use
---
# Define the result
uint32 total_dishes_cleaned
---
# Define a feedback message
float32 percent_complete
```

Request//(client发出的目标请求)

---

Final result Reply//(server发出的回应)

---

Feedback message //(server发出的中间状态信息)



## 7.5 Action

### Action文件格式

**move\_base\_msgs/action/MoveBase.action**

```
geometry_msgs/PoseStamped target_pose  
---  
---  
geometry_msgs/PoseStamped base_position
```

Move\_base 包提供action的实现，在世界中给出一个目标，移动基座将会尝试到达这一点。

MoveBaseAction消息实现多点导航，可从csv文件读取位姿列表





```
wh000@wh000-Lenovo-Ideapad-500S-15ISK: ~/myspace/catkin_ws
wh000@wh000-Lenovo-ideapad-500S-15ISK:~/myspace/catkin_ws$ rosr
shes_server
```

```
wh000@wh000-Lenovo-ideapad-500S-15ISK: ~/myspace/catkin_ws
wh000@wh000-Lenovo-ideapad-500S-15ISK:~/myspace/catkin_ws$ rosr
shes_client
```

```
wh000@wh000-Lenovo-ideapad-500S-15ISK: ~/myspace/catkin_ws

SUMMARY
=====

PARAMETERS
* /rostdistro: kinetic
* /rosversion: 1.12.14

NODES

auto-starting new master
process[master]: started with pid [3663]
ROS_MASTER_URI=http://wh000-Lenovo-ideapad-500S-15ISK:11311/

setting /run_id to 103f3e00-bf93-11eb-8094-00dbdfd81788
process[rosout-1]: started with pid [3676]
started core service [/rosout]
^C[rosout-1] killing on exit
[master] killing on exit
shutting down processing monitor...
... shutting down processing monitor complete
done
wh000@wh000-Lenovo-ideapad-500S-15ISK:~/myspace/catkin_ws$ roscore
```



## 7.5 Action

---

action主要弥补了其他通信方式的不足，或者说各种通信方式的适用场景不同。Action比较适合实现长时间的通信过程，由于通信过程**可以随时被查看具体进度情况**，也可以终止请求，这样使得它在一些特别的机制中拥有很高的效率。



## 小结

---

Topic、Service、Action、Parameter server这四种通信方式组成了ROS的通信基础，不同的通信方式可以满足不同的实际需求。通过对比学习这四种通信方式，去思考每一种通信的优缺点和适用条件，在正确的地方用正确的通信方式，这样整个ROS的通信会更加高效，机器人也将更加的灵活和智能。

对于这些通信方式命令的使用，还可以有效地帮助我们排查问题，能方便相应的机器人功能的开发。

